

Jetpack Compose

Room Database 연동
Hilt 의존성 주입
DataStore 간단 설정 저장

박숙영
blue@sookmyung.ac.kr

도입

- 지금까지 만든 ToDo앱
  - 앱을 종료하면 모든 데이터가 사라짐
  - 즉, 메모리에만 데이터를 저장함(mutableStateListOf, remember 등)

- 데이터를 영구적으로 저장하려면 로컬 DB가 필요

저장소에 데이터 보관하기

- 소량의 환경 설정 (Key - Value)
  - 사용: 다크모드, 알림 on/off, 필터 옵션, 마지막 탭
  - API: DataStore(Preferences/Proto)
    - Preferences: 키-값, 빠르게 쓰기 쉬움
    - Proto: 스키마 기반(타입 안전), 확장성 높음
  - 장점: 비동기/코루틴/Flow 친화적
- 구조화된 데이터(목록/관계/검색) - 로컬 DB
  - 사용: Todo/노트/사진 메타데이터/히스토리 등 CRUD & 쿼리가 필요한 데이터
  - API: Room(SQLite) + Flow
  - 장점: 쿼리/인덱스/관계, 대량 데이터 안정적
  - 주의: 큰 바이너리(이미지/동영상) 자체를 Room에 넣지 말고 경로/URI만 저장

저장소에 데이터 보관하기 (계속)

- 파일/미디어 (바이너리 & 문서)
  - 사용: 이미지, PDF, 오디오 등 대용량 바이너리
  - API: File, MediaStore, Storage Access Framework
  - 장점: 대용량/스트리밍 최적, 공유/백업 용이
  - 주의: Android 10+ Scoped Storage 정책 준수, 퍼미션/권한 흐름 고려
- 원격/동기화 데이터(서버 연동)
  - 사용: 계정 데이터, 팀 데이터, 공용 목록, 업로드/다운로드
  - API: Retrofit(REST)/ Firebase / gRPC + Room 캐시
  - 패턴: offline-first
    - Remote → 캐시(Room) → UI (Compose)
    - 설정(DataStore)로 동기화 규칙 제어

Room vs DataStore

- Room
  - SQL 기반 DB, 로컬 DB 설계, CRUD 중심
  - 구조화된 대량 데이터 관리(예: Todo목록, 사진 정보, 사용자 기록)
  - 앱의 “주요 데이터“ 저장소

- DataStore(Preferences/Proto)
  - Key-Value 기반 단순 설정값 저장(예: 다크 모드 여부, 알림 On/Off)
  - 구조 단순, 직관적
  - 앱의 “환경 설정“ 저장소
  - 사용자 설정/환경 옵션 관리



Jetpack Compose

Room DB

Room DB

SQLite: 내장형 관계형 데이터베이스 관리 시스템, 서버가 아닌 파일 기반으로 작동

- **정의**
  - SQLite 위에 구성된 Jetpack의 객체지향 DB 추상화 라이브러리
- **장점**
  - SQL 작성 최소화, 컴파일 시 쿼리 검증, 코루틴/Flow와 통합 쉬움
- **목적**
  - 앱 내부에 영구 저장소 구성
    - 예) 할 일 목록, 즐겨찾기, 사용자 설정 등

- **Room = SQLite + 타입 안정성 + 구조화된 컴포넌트**

Room 구성 요소

- Entity
  - 테이블에 해당하는 데이터 모델
- DAO
  - SQL 쿼리를 담당하는 인터페이스
- Database
  - RoomDatabase 상속 클래스
- Repository
  - 추상화 계층
- ViewModel 연동
  - LiveData / StateFlow로 UI와 연결

Room을 위한 gradle 설정

```
plugins {
    ...

    id ("kotlin-kapt")
}
```

```
implementation("androidx.room:room-runtime:2.8.2”)
kapt(“androidx.room:room-compiler:2.8.2”)
implementation(“androidx.room:room-ktx:2.8.2")
```

Room – (1) Entity (데이터 클래스)

- Entity는 데이터베이스의 테이블에 해당
- @Entity
  - Room이 SQLite 테이블로 인식
- @PrimaryKey
  - 기본 키 지정
- @ColumnInfo(name = "...")
  - 열 이름 지정 (생략 가능)
- autogenerate = true
  - ID 자동 증가 옵션

- 예

```
@Entity(tableName = "todos")
data class TodoEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,

    @ColumnInfo(name = "title")
    val title: String,

    @ColumnInfo(name = "is_done")
    val isDone: Boolean = false
)
```

```
CREATE TABLE todos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    is_done INTEGER
)
```

Room – (2) DAO (Data Access Object)

- DB에 접근하는 인터페이스
  - 실제 DB 작업을 정의하는 인터페이스
- 쿼리 정의
  - @Insert, @Query, @Delete, @Update 등으로 DB 작업 정의
- 설명
  - getAll()
    - 전체 목록을 Flow로 반환 → UI가 자동으로 갱신
  - @Insert, @Update, @Delete는 모두 suspend
    - 비동기 처리 필수

- 예

```
@Dao
interface TodoDao {
    @Query("SELECT * FROM todos")
    fun getAll(): Flow<List<TodoEntity>>

   @Query("SELECT * FROM todos WHERE is_done = 1")
    fun getCompletedTodos(): Flow<List<TodoEntity>>

    @Insert
    suspend fun insert(todo: TodoEntity)

    @Delete
    suspend fun delete(todo: TodoEntity)

    @Update
    suspend fun update(todo: TodoEntity)
}
```

Room – (3) RoomDatabase

- Room을 사용하려면 반드시 RoomDatabase를 상속한 추상 클래스가 필요
- 실제 DB객체 생성 및 DAO 제공
  - @Database: 테이블 목록과 버전 명시
  - abstract fun todoDao(): DAO 객체 반환 함수 정의, Room이 구현 클래스를 자동 생성
  - 싱글톤 패턴을 통해 DB 인스턴스를 하나만 생성
    - getDatabase()에서 Room.databaseBuilder() 사용
    - 앱 전체에서 하나의 DB 인스턴스를 공유
  - DB를 여러 번 만들면 안 되는 이유: 리소스 낭비, 충돌 가능성

Room – (3) RoomDatabase (계속)

- 예)

```
@Database(entities = [TodoEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun todoDao(): TodoDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "todo_database"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Room – (4) Repository

- 데이터 소스(DAO, 네트워크, 캐시 등)와 ViewModel 사이의 중간 계층
- ViewModel이 데이터 저장소에 직접 접근하지 않도록 추상화
- DB 작업에 대한 비즈니스 로직을 분리해두는 공간
  - SRP(단일 책임 원칙), 관심사 분리(Separation of Concerns) 실현
- 역할과 장점

역할	설명
데이터 소스 추상화	DAO, API, 파일 등 다양한 데이터 소스를 한 인터페이스로 관리
테스트 용이성	테스트 시 가짜 Repository를 주입하여 독립적인 테스트 가능
코드 재사용성	여러 ViewModel 또는 컴포넌트에서 같은 Repository 사용 가능
책임 분리	ViewModel은 UI 로직만 담당, DB 로직은 Repository에 위임

Room – (4) Repository 예제(계속)

- 예제 구조도 [UI] ↔ [ViewModel] ↔ [Repository] ↔ [DAO] ↔ [RoomDB]

계층	책임	주 역할
UI(Composable)	사용자 인터페이스	상태를 구독하고 이벤트를 전달
ViewModel	상태 관리	UI 상태 제공, Repository 호출
Repository	데이터 추상화	DAO에 접근하거나 서버 요청 대리
DAO	SQL 쿼리 정의	DB에 CRUD 실행
RoomDB	영구 저장소	SQLite 기반 내부 저장소

Room – (4) Repository 예제(계속)

[UI] ↔ **[ViewModel]** ↔ **[Repository]** ↔ [DAO] ↔ [RoomDB]

```
class TodoRepository(private val dao: TodoDao) {

    val allTodos: Flow<List<TodoEntity>> = dao.getAll()

    suspend fun insert(todo: TodoEntity) {
        dao.insert(todo)
    }

    suspend fun update(todo: TodoEntity) {
        dao.update(todo)
    }

    suspend fun delete(todo: TodoEntity) {
        dao.delete(todo)
    }
}
```

```
class TodoViewModel(private val repository: TodoRepository) :
ViewModel() {

    val todoList: StateFlow<List<TodoEntity>> =
        repository.allTodos.stateIn(viewModelScope,
                SharingStarted.Lazily, emptyList())

    fun addTodo(todo: TodoEntity) {
        viewModelScope.launch {
            repository.insert(todo)
        }
    }
    fun removeTodo(todo: TodoEntity) {
        viewModelScope.launch {
            repository.delete(todo)
        }
    }
}
```

"만약 나중에 데이터를 로컬 DB가 아닌 서버에서 받아오게 되면, 어디만 바꾸면 좋을까?"
→ Repository만 바꾸면 됨 → **아키텍처 설계의 유연성**

비동기 처리와 코루틴

- "앱에서 할 일을 추가할 때마다 잠깐 멈춘다면? "
- DB에 삽입/수정/삭제하는 작업은 오래 걸릴 수 있음
  - 메인 스레드(UI 스레드)에서 실행하면 앱이 멈추는 현상(ANR 발생)
  - 그래서 Room은 메인스레드에서 DB 작업을 허용하지 않음
  - @Insert, @Update, @Delete 는 반드시 suspend로 선언
  - ViewModel은 viewModelScope.launch {...} 로 **코루틴** 실행

```
fun addTodo(todo: TodoEntity) {
    viewModelScope.launch {
        repository.insert(todo)
    }
}
```

```
fun addTodo(todo: TodoEntity) =
 viewModelScope.launch {
    repository.insert(todo)
}
```

- 사용자 조작(삽입/수정/삭제)은 코루틴으로 처리하여 앱이 끊기지 않게 함

실시간 UI 반영 메커니즘

- "데이터를 추가한 다음 UI는 어떻게 바뀔까?"
- Flow의 역할
  - DAO의 getAll()은 Flow<List<TodoEntity>>로 데이터를 반환
  - ViewModel은 이 Flow를 UI에서 사용할 수 있도록 변환 (collectAsState)
  - DB 데이터 변경 시 → Flow 자동 감지 → Compose가 리컴포지션됨
- 데이터 변경을 감지하고 UI를 새로 그리는 건 Flow와 Compose의 역할

코루틴과 Flow의 연결 구조

사용자 입력
↓
[ViewModel] ← launch 코루틴 (비동기 처리)
↓
[Repository → DAO → DB] ← suspend insert()
↓
DB 변경 발생
↓
[Flow로 감지됨]
↓
[ViewModel 상태 갱신]
↓
[UI가 collectAsState로 감지]
↓
UI 자동 업데이트 (리컴포지션)



Jetpack Compose

Hilt 의존성 주입

박숙영
blue@sookmyung.ac.kr

Hilt

- Android에서 DI(Dependency Injection, 의존성 주입)를 쉽게 구현하기 위한 Jetpack 라이브러리
- @HiltViewModel과 hiltViewModel()을 통해 의존성 주입이 완료된 ViewModel을 Navigation에 맞춰 자동 생성해줌
- 주요 기능

기능	설명
@HiltViewModel	ViewModel에 의존성을 자동 주입
@Inject	생성자 기반 주입 선언
@Module + @Provides	의존성 생성 방법을 명시
@AndroidEntryPoint	의존성 주입이 필요한 Activity, Fragment, Composable 진입점 지정
@HiltAndroidApp	Application 클래스에 선언하여 Hilt를 앱 전체에 초기화

AppModule

- Hilt에서 의존성 주입을 “어떻게“, “무엇을“, “어디에“ 제공할지를 정의하는 설정 클래스
- 앱 전역에서 재사용 가능한 객체들을 Hilt Container에 등록하는 장소
- AppModule의 역할

역할	설명
의존성 제공자 역할	@Provides 함수를 통해 필요한 객체를 생성하고 Hilt에 등록
싱글톤 관리	@Singleton으로 앱 전역에 동일한 인스턴스를 사용하도록 설정 가능
객체 생성 로직 분리	ViewModel, Activity, Composable 등에서 객체를 직접 생성하지 않게 해줌 (의존성 역전)
테스트 용이성 향상	객체 생성이 AppModule에 모여 있어 테스트에서 쉽게 Mock으로 대체 가능

Hilt의 장점

- 의존성 관리의 자동화
  - RoomDatabase, DAO, Repository 등은 서로 의존 관계
  - Hilt를 사용하면 각 객체를 직접 생성할 필요 없이 자동으로 주입
- 구조의 분리와 테스트 용이성
  - Hilt는 느슨한 결합(Loosely Coupled) 구조를 유도
  - Repository나 Dao를 테스트 할 때 Mock 객체로 교체하기 쉬움
- 전역에서 안전하게 인스턴스 재사용 가능
  - Hilt에서 @Singleton 스코프를 통해 앱 전역에서 하나의 DB 인스턴스를 공유
- 코드 간결화
  - 생성자에 @Inject만 붙이면 Hilt가 알아서 필요한 객체를 제공

AppModule 예시

앱 전역에 필요한 객체들을 Hilt에 등록하고,
ViewModel 등에서 이를 자동 주입받게 해주는 연결고리

```
@Module                                          // 이 클래스가 Hilt에 의존성을 제공함을 알림
@InstallIn(SingletonComponent::class)            // 앱 전역에서 사용할 수 있게 함
object AppModule {
    // Room DB 인스턴스 제공
    @Provides
    @Singleton                                   // 싱글 인스턴스(전역 단 하나)로 유지되도록 설정
    fun provideDatabase(@ApplicationContext context: Context): AppDatabase {
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "todo_database"
        ).build()
    }
    // TodoDao 제공
    @Provides
    fun provideTodoDao(database: AppDatabase): TodoDao {
        return database.todoDao()
    }
    // Repository 제공
    @Provides
    fun provideTodoRepository(todoDao: TodoDao): TodoRepository {
        return TodoRepository(todoDao)
    }
}
```

실습 예제

- ToDo 앱에 Hilt 주입, Room DB 연동하여 CRUD 구현

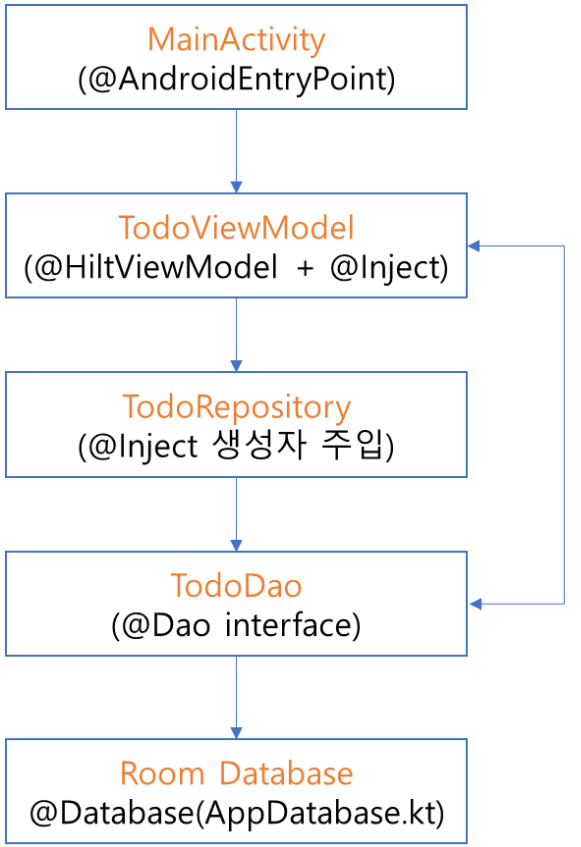
- 프로젝트 생성: TodoApplication

- 기능
  - 할 일 추가하면 DB 에 저장
  - 앱 종료 후에도 데이터 유지
  - 수정 버튼으로 내용 변경
  - 삭제 버튼으로 항목 제거
  - 목록 자동 리컴포지션

프로젝트 구조

- data/
  - TodoEntity.kt // @Entity
  - TodoDao.kt // @Dao
  - AppDatabase.kt // @Database
- di/
  - AppModule.kt // @Module + @Provides로 DB, DAO 제공
- navigation/
  - NavGraph.kt
- repository/
  - TodoRepository.kt // @Inject constructor
- viewmodel/
  - TodoViewModel.kt // @HiltViewModel + @Inject constructor
- screens/
  - TodoListScreen.kt // 목록 화면
  - AddEditTodoScreen.kt // 추가/수정 화면
- MainActivity.kt // @AndroidEntryPoint – ViewModel 주입 가능
- MyApp.kt // @HiltAndroidApp – 앱 전역 Hilt 활성화

▪ 전체 구성 흐름도



build.gradle 설정 (수정)

- VersionCatalog (libs.versions.toml)
  - versions
  - libraries
    - Lifecycle/ViewModel
    - Navigation Compose
    - Room
    - Hilt
  - plugins

```
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
hilt = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
```

- build.gradle.kts (Project)
  - plugins

```
alias(libs.plugins.ksp) apply false
alias(libs.plugins.hilt) apply false
```

- build.gradle.kts (App)
  - plugins

```
alias(libs.plugins.ksp)
alias(libs.plugins.hilt)
```

build.gradle 설정 (수정)

kapt = Kotlin Annotation Processing Tool (이전 버전)
KSP = Kotlin Symbol Processing)

- build.gradle.kts (App)
  - dependencies

- implementation은 라이브러리를 사용하기 위한 것
- ksp는 코틀린 전용 코드 생성용 프로세서 프레임워크
  '@'이 붙은 코드를 찾아 그 정보 기반 자동으로 Kotlin 코드 생성

```
// Lifecycle /Viewmodel
implementation(libs.androidx.lifecycle.viewmodel)
implementation(libs.androidx.lifecycle.runtime.ktx)

// Navigation(Compose)
implementation(libs.androidx.navigation.compose)

// Room dependencies
implementation(libs.androidx.room.runtime)
ksp(libs.androidx.room.compiler)
implementation(libs.androidx.room.ktx)

// Hilt  dependencies
implementation(libs.hilt.android)
ksp(libs.hilt.compiler)
implementation(libs.hilt.lifecycle.vm.compose)
```

Hilt 의존성 주입의 시작점

- @HiltAndroidApp
  - 앱의 의존성 그래프를 초기화하고 Hilt 주입 기능을 사용 가능하게 해주는 Application 클래스 어노테이션
  - MyApp.kt

```
@HiltAndroidApp
class MyApp: Application()
```

- Manifests 파일에 앱 등록

```
<application
    android:name = ".MyApp"
```

  - 등록하지 않으면 오류 발생

    Hilt Activity must be attached to an @HiltAndroidApp Application

/data/TodoEntity, TodoDao, AppDatabase

```kotlin
@Entity(tableName = "todos")
data class TodoEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val tag: String = "", // 태그
    val memo: String = "", // 메모
    val description: String = "", // 설명
    val isDone: Boolean = false // 완료  여부
)
```

```kotlin
@Dao
interface TodoDao {
    @Query("SELECT * FROM todos")
    fun getAll(): Flow<List<TodoEntity>>

    @Insert
    suspend fun insert(todo: TodoEntity)

    @Update
    suspend fun update(todo: TodoEntity)

    @Delete
    suspend fun delete(todo: TodoEntity)
}
```

```kotlin
@Database(entities = [TodoEntity::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun todoDao(): TodoDao

    companion object {
        @Volatile private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "todo_db"
                ).build().also { INSTANCE = it }
            }
        }
    }
}
```

/repository/TodoRepository

```
class TodoRepository @Inject constructor(private val dao: TodoDao) {
    val allTodos = dao.getAll()

    suspend fun insert(todo: TodoEntity) = dao.insert(todo)
    suspend fun update(todo: TodoEntity) = dao.update(todo)
    suspend fun delete(todo: TodoEntity) = dao.delete(todo)
}
```

/viewmodel/TodoViewModel

```
@HiltViewModel
class TodoViewModel @Inject constructor(
    private val repository: TodoRepository
) : ViewModel() {
    val todoList: StateFlow<List<TodoEntity>> =
        repository.allTodos.stateIn(viewModelScope, SharingStarted.Lazily, emptyList())

    fun addTodo(todo: TodoEntity) {
        viewModelScope.launch {
            repository.insert(todo)
        }
    }
    fun updateTodo(todo: TodoEntity) {
        viewModelScope.launch {
            repository.update(todo)
        }
    }
    fun deleteTodo(todo: TodoEntity) {
        viewModelScope.launch {
            repository.delete(todo)
        }
    }
}
```

/di/AppModule

```
@Module
@InstallIn(SingletonComponent::class) // 앱 전역에서 사용할 수 있게 함
object AppModule {
    // Room DB 인스턴스 제공
    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): AppDatabase {
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "todo_database"
        ).build()
    }
    // TodoDao 제공
    @Provides
    fun provideTodoDao(database: AppDatabase): TodoDao {
        return database.todoDao()
    }
    // Repository 제공
    @Provides
    fun provideTodoRepository(todoDao: TodoDao): TodoRepository {
        return TodoRepository(todoDao)
    }
}
```

/screens/TodoListScreen

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TodoListScreen(viewModel: TodoViewModel, navController: NavController) {
    val todos by viewModel.todoList.collectAsState()
    Scaffold(
        topBar = { TopBar() },
        floatingActionButton = {
            FloatingActionButton(onClick = { navController.navigate("addEdit") }) {
                Icon(
                    imageVector = Icons.Default.Add,
                    contentDescription = "Add Todo"
                )
            }
        }
    ) { padding ->
        LazyColumn(modifier = Modifier.padding(padding)) {
            items(todos) { todo ->
                TodoItem(
                    todo = todo,
                    viewModel = viewModel,
                    navController = navController
                )
            }
        }
    }
}
```

/screens/TopBar

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TopBar(){
    TopAppBar(
        title = { Text("Todo List") },
        actions = {
            IconButton(onClick = { /* Handle settings click */ }) {
                Icon(Icons.Default.MoreVert,  contentDescription = "Settings")
            }
        }
    )
}
```

- 설정 버튼의 기능 추후 추가 예정

/screens/TodoItem

```
@Composable
fun TodoItem(todo: TodoEntity, viewModel: TodoViewModel, navController: NavController) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable {
                navController.navigate("addEdit?id=${todo.id}")
            }
            .padding(16.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(8.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Checkbox(checked = todo.isDone, onCheckedChange = { isChecked ->
                val updatedTodo = todo.copy(isDone = isChecked)
                viewModel.updateTodo(updatedTodo)
            })
```

/screens/TodoItem (계속)

```
Column(
            modifier = Modifier
                .weight(1f)
                .padding(start = 8.dp)
        ) {
            Text(text = todo.title, fontSize = 16.sp, modifier = Modifier.padding(start = 8.dp))
            Text(todo.memo, modifier = Modifier.padding(start = 8.dp), fontSize = 10.sp)
        }
        Text(todo.tag, modifier = Modifier.padding(start = 8.dp), fontSize = 10.sp)
        IconButton(onClick = { navController.navigate("addEdit?id=${todo.id}") }) {
            Icon(
                imageVector = Icons.Default.Edit,
                contentDescription = "Edit Todo"
            )
        }
        IconButton(onClick = { viewModel.deleteTodo(todo) }) {
            Icon(
                imageVector = Icons.Default.Delete,
                contentDescription = "Delete Todo"
            )
        }
    }
  }
}
```

/screens/AddEditTodoScreen

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AddEditTodoScreen(
    viewModel: TodoViewModel,
    navController: NavController,
    backStackEntry: NavBackStackEntry
) {
    val todoId = backStackEntry.arguments?.getString("id")?.toIntOrNull()
    val todoList by viewModel.todoList.collectAsState()
    val existingTodo = todoList.find { it.id == todoId }

    var text by remember { mutableStateOf(existingTodo?.title ?: "") }
    var tag by remember { mutableStateOf(existingTodo?.tag ?: "") }
    var memo by remember { mutableStateOf(existingTodo?.memo ?: "") }

    Scaffold(
        topBar = { TopBar() }
    ) { padding ->
        Column(modifier = Modifier
            .padding(padding)
            .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
```

```
TextField(
            modifier = Modifier.fillMaxWidth(),
            value = text,
            onValueChange = { text = it },
            label = { Text("할  일") })
        TextField(
            modifier = Modifier.fillMaxWidth(),
            value = tag,
            onValueChange = { tag = it },
            label = { Text("태그") })
        TextField(
            modifier = Modifier.fillMaxWidth(),
            value = memo,
            onValueChange = { memo = it },
            label = { Text("메모") })

        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(20.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            //... 뒷 슬라이드
```

/screens/AddEditTodoScreen

```
val newOrUpdatedTodo = TodoEntity(
    id = todoId ?: 0,
    title = text,
    tag = tag,
    memo = memo
)
Button(onClick = {
    if (todoId != null) {
        viewModel.updateTodo(newOrUpdatedTodo)
    } else {
        viewModel.addTodo(newOrUpdatedTodo)
    }
    navController.popBackStack()
}) {
    if (todoId != null) {
        Text("수정")
    } else {
        Text("추가")
    }
}
if (todoId != null) {
    Button(onClick = {
        viewModel.deleteTodo(newOrUpdatedTodo)
        navController.popBackStack()
    }) {
        Text("삭제")
    }            }        }      }   }}
```

/navigation/NavGraph

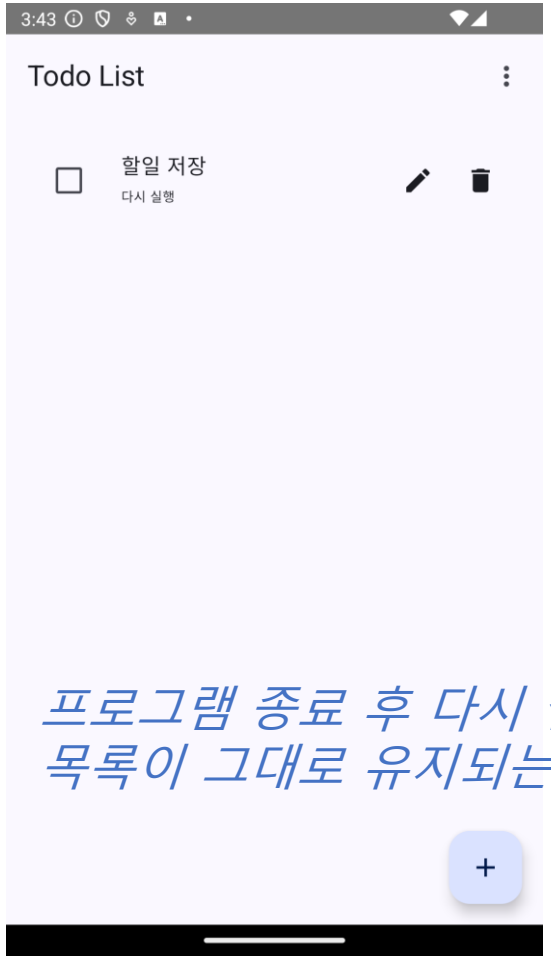
```
@Composable
fun NavGraph(navController: NavHostController) {
    val viewModel: TodoViewModel = hiltViewModel()

    NavHost(navController = navController, startDestination = "list") {
        composable("list") {
            TodoListScreen(viewModel, navController)
        }
        composable("addEdit?id={id}") { backStackEntry ->
            AddEditTodoScreen(viewModel, navController, backStackEntry)
        }
        composable("addEdit") { backStackEntry ->
            AddEditTodoScreen(viewModel, navController, backStackEntry)
        }
    }
}
```

MainActivity

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            FirstApplicationTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    val navController = rememberNavController()
                    NavGraph(navController = navController)
                }
            }
        }
    }
}
```



프로그램 종료 후 다시 실행해도
목록이 그대로 유지되는 것을 확인할 수 있음

- FirstApplicationTheme → 각자 프로젝트의 Theme로 변경하면 됨



Jetpack Compose

DataStore

박숙영
blue@sookmyung.ac.kr

SharedPreferences

- Key-Value 기반 영속 저장소
- 앱 내부 파일로 저장, 앱 종료 후에도 데이터 유지
- 단순 플래그/설정에 적합

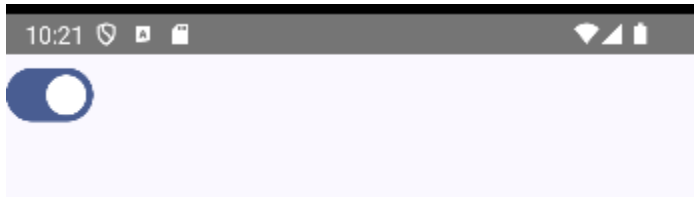
- 단점
  - 동기식 API → Blocking I/O (메인스레드 차단 위험)
  - 타입 안정성 부족(String key 기반 → 오타 위험)
  - LiveData/Flow 같은 반응형 스트림과 통합 어려움

SharedPreferences 예시

```
val context = LocalContext.current
val prefs = context.getSharedPreferences("settings", Context.MODE_PRIVATE)
var darkMode by remember { mutableStateOf(prefs.getBoolean("dark_mode", false)) }

Switch(
    checked = darkMode,
    onCheckedChange = { checked ->
        darkMode = checked
        prefs.edit().putBoolean("dark_mode", checked).apply()
    }
)
```





Jetpack DataStore

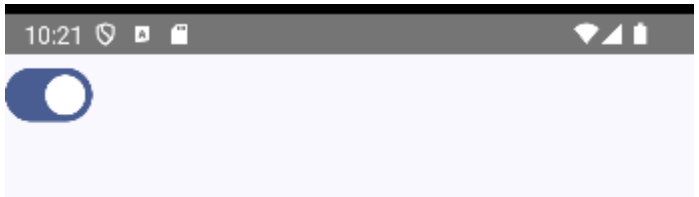
- Jetpack에서 권장하는 최신 Key-Value 저장
- Coroutine + Flow 기반, 비동기 안전
- 두 가지 방식 제공
  - Preferences DataStore
    - Key-Value
    - 타입 안전성 낮음 → 학습용 /간단 옵션
  - Proto DataStore
    - Schema 기반(Protocol Buffer)
    - 타입 안정성 높음 → 확장성 좋음

DataStore 예시

- build.gradle 설정

```
implementation("androidx.datastore:datastore-preferences:1.1.1")
```





- 코틀린

```
// 클래스 외부에 선언
val Context.dataStore by preferencesDataStore("settings")
val DARK_MODE = booleanPreferencesKey("dark_mode")
```

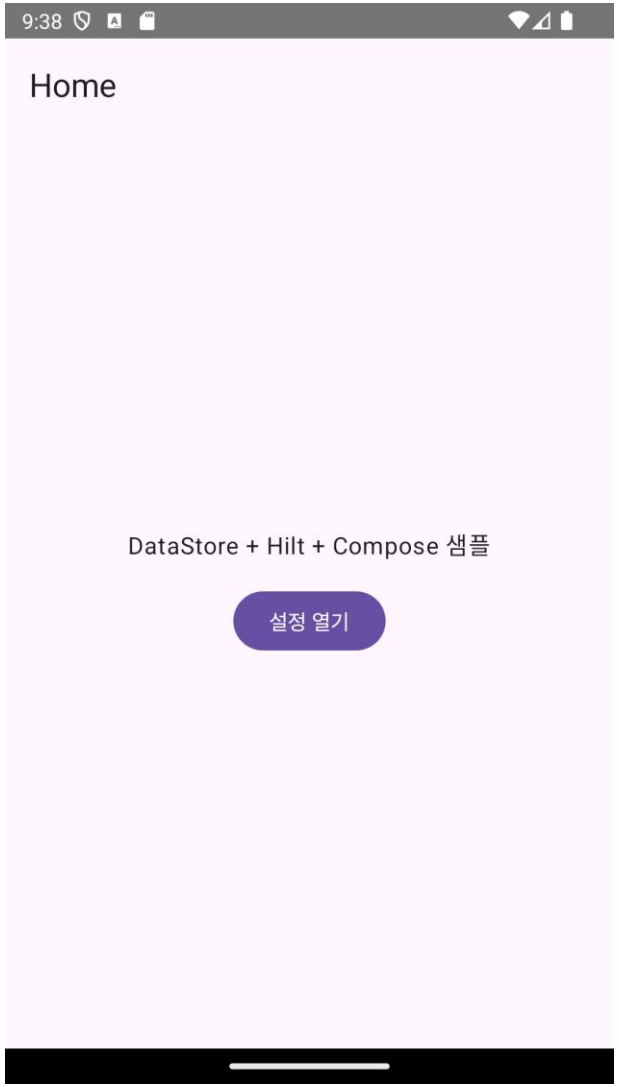
```
val scope = rememberCoroutineScope()
val context = LocalContext.current

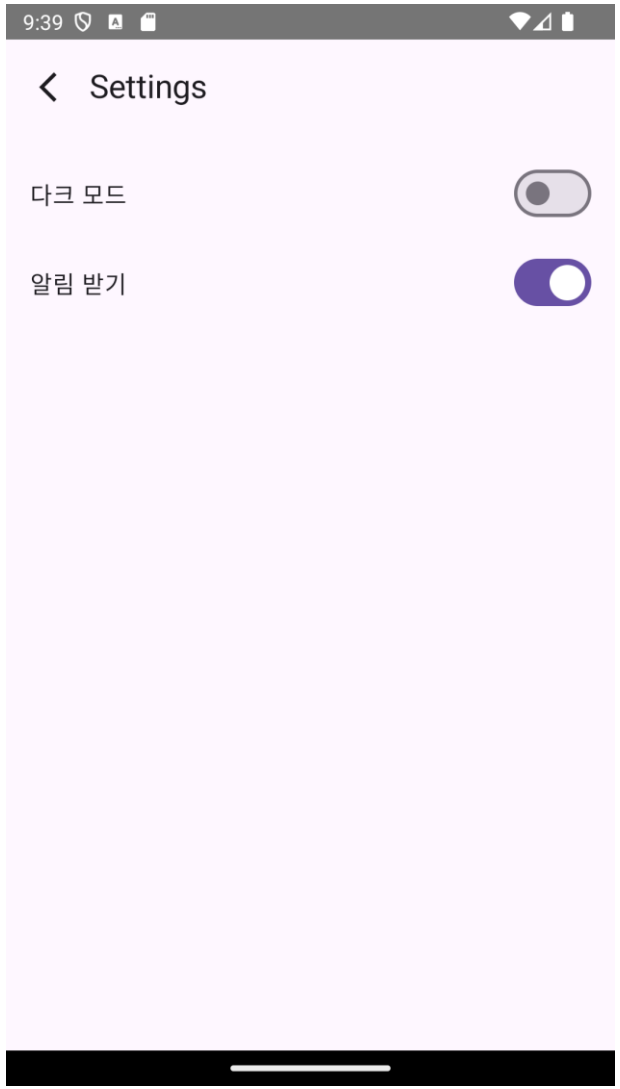
val darkFlow = context.dataStore.data.map { it[DARK_MODE] ?: false }    // 읽기

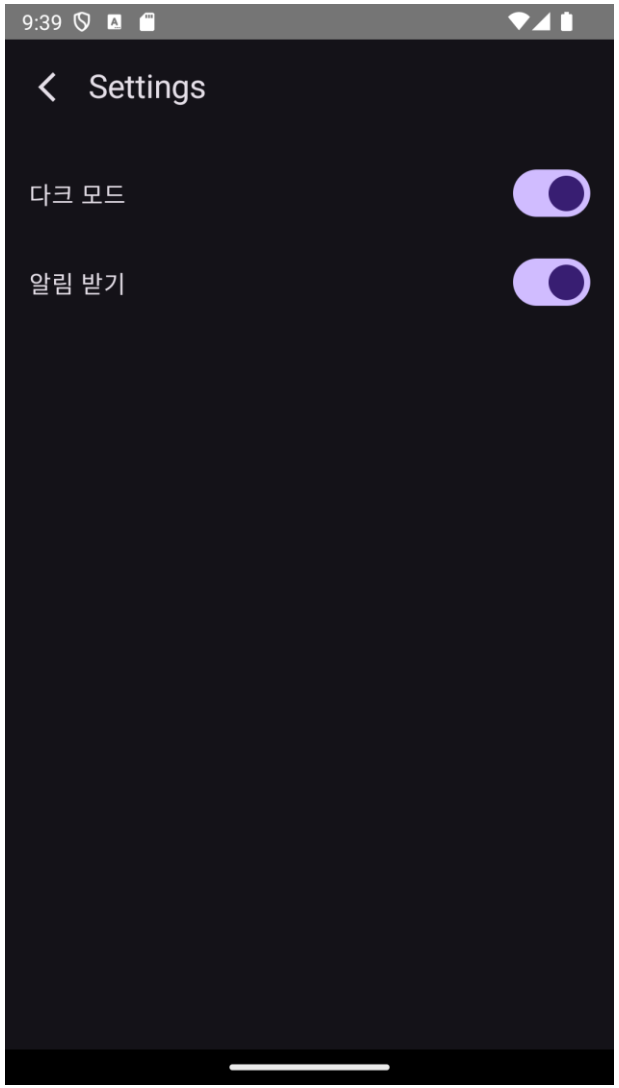
suspend fun setDark(context: Context, value: Boolean) {
    context.dataStore.edit { it[DARK_MODE] = value }
}
// Compose 에서
val dark by darkFlow.collectAsState(initial = false)

Switch(checked = dark, onCheckedChange = {checked ->    scope.launch { setDark(context, checked)  }  } )
```

DataStore 활용 예시







build.gradle.kts

```
plugins {
    ...
    id ("com.google.dagger.hilt.android")
    id ("kotlin-kapt")
}
```

```
...
// Navigation for Compose
implementation("androidx.navigation:navigation-compose:2.7.0")
implementation("androidx.hilt:hilt-navigation-compose:1.2.0")

// DataStore (Preferences)
implementation("androidx.datastore:datastore-preferences:1.1.1")

// Hilt
implementation("com.google.dagger:hilt-android:2.51.1")
kapt("com.google.dagger:hilt-android-compiler:2.51.1")

// Coroutines
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3“)
```

매니페스트 파일 및 앱(Hilt App) 작성

- AndroidManifest.xml

```
<application
    android:name = ".App"
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.FirstApplication">
    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.FirstApplication">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

- App.kt

```
import android.app.Application
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class App : Application()
```

di/DataStoreModule.kt

```
@Module
@InstallIn(SingletonComponent::class)
object DataStoreModule {

    @Provides
    @Singleton
    fun providePreferencesDataStore(
        @ApplicationContext context: Context
    ): DataStore<Preferences> = PreferenceDataStoreFactory.create(
        produceFile = { context.preferencesDataStoreFile("settings") }
    )
}
```

data/settings/SettingsRepository.kt

```
private object SettingsKeys {
    val DARK_MODE = booleanPreferencesKey("dark_mode")
    val NOTIFICATION = booleanPreferencesKey("notification")
}

@Singleton
class SettingsRepository @Inject constructor(
    private val dataStore: DataStore<Preferences>
) {
    val darkModeFlow: Flow<Boolean> = dataStore.data.map { it[SettingsKeys.DARK_MODE] ?: false }
    val notificationFlow: Flow<Boolean> = dataStore.data.map { it[SettingsKeys.NOTIFICATION] ?: true }

    suspend fun setDarkMode(value: Boolean) {
        dataStore.edit { it[SettingsKeys.DARK_MODE] = value }
    }
    suspend fun setNotification(value: Boolean) {
        dataStore.edit { it[SettingsKeys.NOTIFICATION] = value }
    }
}
```

ui/theme/AppTheme.kt

```
private val LightColors = lightColorScheme()
private val DarkColors = darkColorScheme()

@Composable
fun AppTheme(darkTheme: Boolean = isSystemInDarkTheme(), content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = if (darkTheme) DarkColors else LightColors,
        typography = MaterialTheme.typography,
        content = content
    )
}
```

ui/theme/ThemeViewModel.kt

```
@HiltViewModel
class ThemeViewModel @Inject constructor(
    repo: SettingsRepository
) : ViewModel() {
    // 다크모드 플래그를 최상단에서 관찰해 테마에 반영
        val isDark = repo.darkModeFlow
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), false)
}
```

ui/navigation/AppNav.kt

```
object Routes {
    const val HOME = "home"
    const val SETTINGS = "settings"
}

@Composable
fun AppNav(modifier: Modifier = Modifier, navController: NavHostController = rememberNavController()) {
    NavHost(navController = navController, startDestination = Routes.HOME, modifier = modifier) {
        composable(Routes.HOME) {
            HomeScreen(onOpenSettings = { navController.navigate(Routes.SETTINGS) })
        }
        composable(Routes.SETTINGS) {
            val vm: SettingsViewModel = hiltViewModel()
            SettingsScreen(vm = vm, onBack = { navController.popBackStack() })
        }
    }
}
```

ui/screens/HomeScreen.kt

```
@Composable
fun HomeScreen(onOpenSettings: () -> Unit) {
    Scaffold(
        topBar = { TopAppBar(title = { Text("Home") }) }
    ) { p ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(p)
                .padding(16.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text("DataStore + Hilt + Compose 샘플")
            Spacer(Modifier.height(16.dp))
            Button(onClick = onOpenSettings) { Text("설정  열기") }
        }
    }
}
```

ui/settings/SettingsViewModel.kt

```
@HiltViewModel
class SettingsViewModel @Inject constructor(
    private val repo: SettingsRepository
) : ViewModel() {

    val darkMode = repo.darkModeFlow
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), false)

    val notification = repo.notificationFlow
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), true)

    fun toggleDarkMode(v: Boolean) = viewModelScope.launch { repo.setDarkMode(v) }
    fun toggleNotification(v: Boolean) = viewModelScope.launch { repo.setNotification(v) }
}
```

ui/screens/SettingsScreen.kt

```
@Composable
fun SettingsScreen(
    vm: SettingsViewModel = hiltViewModel(),
    onBack: () -> Unit
) {
    val dark = vm.darkMode.collectAsStateWithLifecycle().value
    val notif = vm.notification.collectAsStateWithLifecycle().value

    Scaffold(
        topBar = {
            TopAppBar(title = { Text("Settings") },  navigationIcon = { IconButton(onClick = onBack) {Icon(Icons.Filled.ArrowBack, contentDescription = "Back")  }   }   )
        }
    ) { p ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(p)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth()) {
                Text("다크  모드", modifier = Modifier.weight(1f))
                Switch(checked = dark, onCheckedChange = vm::toggleDarkMode)
            }
            Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth()) {
                Text("알림  받기", modifier = Modifier.weight(1f))
                Switch(checked = notif, onCheckedChange = vm::toggleNotification)
            }
        }
    }
}
```

MainActivity.kt

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val themeVM: ThemeViewModel = hiltViewModel()
            val dark by themeVM.isDark.collectAsStateWithLifecycle()

            AppTheme(darkTheme = dark) {
                AppNav()
            }
        }
    }
}
```